Cash Receipt Feature — Implementation & Debugging Reference

Date: February 20, 2026
Status: Complete and Working
Files Modified: `supply.html`, `scripts.js`, `admin.html`, `supabase-service.js`
Files Created: `cash-receipt-database-deployment.sql`, `add-cash-receipt-permission.sql`

Overview

This feature tracks all cash payment receipts from two sources:

- **Supply bills** — from `approved_bills` table (payment_method = 'Cash')
- **Ugrai payments** — from `ugrai_payment_history` table (full and partial cash payments)

Users can view all cash payments, filter by date/status, and add or update Tally receipt numbers.

Architecture

Database Layer

- `cash_receipts` table — stores tally receipt numbers linked to payments
- `get_all_cash_payments(p_start_date, p_end_date)` — PostgreSQL function using `UNION ALL` to merge supply and ugrai payments
- RLS policies — restrict access to `admin` and `accountant` roles

Backend (`supabase-service.js`)

- `getAllCashPayments(startDate, endDate)` — calls the DB function
- `submitCashReceipt(receiptData)` — upserts into `cash_receipts` table

Frontend (`scripts.js`)

- `initializeCashReceiptDashboard()` — DOM element lookup, date defaults, event listener setup
- `loadCashPayments()` — fetches data, applies filters, renders table
- `renderCashReceiptsTable()` — generates table HTML with pagination
- `applyStatusFilter()` — filters allPayments into filteredPayments
- `handleUpdateReceipt()` — prompts for receipt number and submits
- `exportCashReceiptsToCSV()` — downloads filtered data as CSV

RBAC

- Permission: `cash_receipt.view`
- Roles with access: `admin`, `accountant`
- Tab is hidden/shown via `showCashReceiptTabIfAuthorized()` based on this permission

Problems Faced and How They Were Fixed

Problem 1: SQL Type Mismatch — UUID vs INTEGER

Error:

```
ERROR: operator does not exist: uuid = integer
```

Root Cause:
`approved_bills.id` is `INTEGER`, but `ugrai_payment_history.id` is `UUID`. The `UNION ALL` query tried to merge them into a single column without explicit casting.

Fix:
Cast everything to `TEXT` so the column type is consistent across both branches of the `UNION ALL`:

```
ab.id::TEXT AS id,
uph.id::TEXT AS id,
```

Lesson: When using `UNION ALL` across tables with different ID types, always cast to `TEXT`. Never assume ID column types match across tables.

Problem 2: Wrong Column Name in SQL Function

Error:

```
ERROR: column "received_amount" does not exist
```

Root Cause:
The column in `ugrai_payment_history` is named `outstanding_amount`, not `received_amount`. Wrong column name was used in the query.

Fix:

```
-- Wrong:
COALESCE(uph.received_amount, 0) > 0

-- Correct:
COALESCE(uph.outstanding_amount, 0) > 0
```

Lesson: Always verify exact column names with `\d table_name` in psql or check the schema file before writing SQL queries against unfamiliar tables.

Problem 3: VARCHAR Cannot Be Cast to TEXT Implicitly in UNION

Error:

```
ERROR: UNION types character varying and text cannot be matched
```

Root Cause:
Some columns in `approved_bills` were `VARCHAR` type. PostgreSQL's `UNION ALL` requires matching types. `VARCHAR` and `TEXT` are not automatically coerced.

Fix:
Explicit `::TEXT` cast on all `VARCHAR` columns:

```
ab.retailer_name::TEXT,
ab.sector::TEXT,
ab.payment_method::TEXT
```

Lesson: In `UNION ALL` queries, cast every column explicitly to its target type. Don't rely on implicit coercion, even between closely related types like `VARCHAR` and `TEXT`.

Problem 4: TIMESTAMP Cannot Be Cast to DATE in UNION

Error:

```
ERROR: UNION types timestamp without time zone and date cannot be matched
```

Root Cause:
`approved_bills.approval_date` is `TIMESTAMP`, but the target column was expected as `DATE`. PostgreSQL won't implicitly truncate a timestamp to a date in a UNION.

Fix:

```
ab.approval_date::DATE AS payment_date
```

Lesson: When building a unified view from multiple tables, define the target type for every column upfront and cast all source columns explicitly to that type.

Problem 5: RBAC Permission Not Added

Symptom:
After deploying the database, the Cash Receipt tab was not visible for admin users.

Root Cause:
The `cash_receipt.view` permission was never added to any role in the `roles.permissions` JSONB array.

Fix (`add-cash-receipt-permission.sql`):

```
UPDATE roles
SET permissions = permissions || '["cash_receipt.view"]'::jsonb
WHERE name IN ('admin', 'accountant')
AND NOT (permissions ? 'cash_receipt.view');
```

The `NOT (permissions ? 'cash_receipt.view')` guard prevents duplicate entries on repeated runs.

Lesson: When adding a new feature with RBAC, always create and run the permission SQL as part of the deployment checklist before testing the UI.

Problem 6: Content Opacity Stuck at 0.5 (Invisible Rendered Table)

Symptom:
Console showed 233 payments loaded, 25 rows rendered, 33KB HTML in the DOM — but screen was completely blank.

Root Cause:
`loadCashPayments()` set `content.style.opacity = '0.5'` during loading. The `finally` block was supposed to reset it to `1`, but:

1. A race condition caused the function to be called twice simultaneously (once by the main tab handler, once by the click event listener)
2. The second concurrent call reset opacity to `0.5` again after the first call's `finally` block ran
3. There was no loading guard to prevent concurrent calls

Fix:

```
let isLoadingPayments = false;

async function loadCashPayments() {
    if (isLoadingPayments) {
        console.log('⏳ Already loading, skipping duplicate call');
        return;
    }
    isLoadingPayments = true;
    try {
```

```
            // ... load and render
        } finally {
            if (cashReceiptElements.content) {
                cashReceiptElements.content.style.opacity = '1';
                cashReceiptElements.content.style.pointerEvents = 'auto';
            }
            isLoadingPayments = false;
        }
    }
```

Lesson: Any async data-loading function that modifies visual state (opacity, display, classes) must have a loading guard flag to prevent concurrent calls from corrupting the UI state.

Problem 7: Dashboard Container Outside `<main>` (Blank Screen)

Symptom:
Even after fixing the opacity issue, the screen was completely blank. Console confirmed data was loaded and table rows were in the DOM.

Root Cause:
The `cash-receipt-dashboard-container` div was placed **after** the closing `</main>` tag in `supply.html`. The `<main id="main-content-container">` ends at line 2497. The cash receipt HTML was sitting at line 2556 — outside the visible scrollable content area entirely.

All other dashboard containers (supply, ugrai, purchase, analysis) are **inside** `<main>`. The cash receipt container was accidentally placed outside it, likely appended at the bottom of the file without checking the main tag boundary.

Fix:
Moved the entire `cash-receipt-dashboard-container` block to **inside** `<main>`, as a sibling to the other dashboard containers, just before `</main>`.

How to verify placement is correct:
Run this in terminal:

```
Select-String -Path "supply.html" -Pattern "</main>" -SimpleMatch | Select-Object -First 1
```

The line number returned must be **after** the line numbers of all dashboard containers.

Lesson: When adding a new dashboard container to `supply.html`, always check the `</main>` tag line number first. All dashboard containers must live inside `<main id="main-content-container">`. This is a non-obvious structural requirement because `renderMainContent()` uses `classList.remove('hidden')` — it will succeed even if the element is outside `<main>`, but nothing visible will happen.

Problem 8: Sticky Table Header Bleeding Over Navbar on Scroll

Symptom:
When scrolling down in the Cash Receipt table, the sticky `<thead>` row would visually overlap the top navbar.

Root Cause:
Both the app navbar (`<header class="sticky top-0 z-10">`) and the table `<thead class="sticky top-0 z-10">` had the same `z-10` stacking context. When the page scrolled, the thead competed with the navbar at the same z-level.

Fix:
Lower the thead's z-index to `z-[1]` (Tailwind arbitrary value):

```
<!-- Before -->
<thead class="... sticky top-0 z-10">

<!-- After -->
<thead class="... sticky top-0 z-[1]">
```

The thead only needs to be sticky within its own `max-h-[600px] overflow-y-auto` scroll container, so a lower z-index is correct. The navbar at `z-10` will always render on top.

Lesson: Table `<thead sticky>` elements should never have the same z-index as page-level sticky navbars. Use `z-[1]` for table headers that scroll within a bounded container.

Problem 9: `handleUpdateReceipt` Not Defined (ES Module Scope)

Error:

```
Uncaught ReferenceError: handleUpdateReceipt is not defined
    onclick http://127.0.0.1:5500/supply.html:1
```

Root Cause:
`scripts.js` uses ES module syntax (`import` statements at the top). In ES modules, all top-level declarations are **module-scoped**, not global. Inline `onclick="handleUpdateReceipt(...)"` attributes in dynamically generated HTML look for the function on the global (`window`) scope — where it doesn't exist.

Fix:
Explicitly expose the function on `window` after its declaration:

```
async function handleUpdateReceipt(paymentId, currentReceipt, referenceNumber)
{
    // ... implementation
```

```
}

// Must be global for inline onclick handlers (ES module limitation)
window.handleUpdateReceipt = handleUpdateReceipt;
```

Lesson: In any project using ES modules (`type="module"` or `import` syntax), **no** function is automatically global. Any function called from an inline HTML `onclick`, `onchange`, etc. attribute must be explicitly assigned to `window`. Prefer `addEventListener` in JS code over inline `onclick` attributes to avoid this entirely.

Problem 10: `supabase` Redeclaration Error in admin.html

Error:

```
Uncaught SyntaxError: redeclaration of non-configurable global property
supabase
    <anonymous> http://127.0.0.1:5500/admin.html:772
```

Root Cause:
Two issues compounded:

1. The Supabase CDN script (`@supabase/supabase-js@2`) attaches itself to `window.supabase` as a **non-configurable** property
2. The inline script then declared `const supabase = window.supabase.createClient(...)` — attempting to shadow the same name as a block-scoped `const`, which the browser rejects

Additionally, stray text `y might not be loading into the Analysis section because:` had been accidentally left between two `<script>` tags, breaking the HTML structure.

Fix:

1. Removed the stray text between script tags
2. Renamed `const supabase` → `const supabaseClient`
3. Updated all 3 references: `new RBACService(supabase)`, `await supabase.from(...)`, `await supabase.auth.signOut()`

```
// Before (causes error)
const supabase = window.supabase.createClient(...);

// After (safe)
const supabaseClient = window.supabase.createClient(...);
```

Lesson: Never name a local variable the same as a CDN library's global export. Supabase CDN always claims `window.supabase`. Always use `supabaseClient` or a similarly distinct name. Also, be careful when

editing between `<script>` tags — stray text outside tags can silently corrupt the HTML and cause parsing errors.

Deployment Checklist (for future reference)

When deploying this feature to a new environment or after a database reset:

1. **Run** `cash-receipt-database-deployment.sql` — creates table, indexes, RLS policies, trigger, and `get_all_cash_payments()` function
2. **Run** `add-cash-receipt-permission.sql` — adds `cash_receipt.view` to admin and accountant roles
3. **Verify** the permission was added:

```
SELECT name, permissions FROM roles WHERE name IN ('admin',
'accountant');
```

4. **Hard refresh** the browser (Ctrl+F5) to clear any cached JS

Key Technical Constraints to Remember

Constraint	Detail
Supply bill IDs	`INTEGER` type in `approved_bills`
Ugrai payment IDs	`UUID` type in `ugrai_payment_history`
Unified ID column	Must be `TEXT` to hold both
`cash_receipts.payment_reference_id`	`TEXT NOT NULL` — intentionally accepts both formats
`get_all_cash_payments()`	Must cast all columns explicitly — no implicit coercion in UNION ALL
Dashboard container placement	Must be inside `<main id="main-content-container">`
Inline onclick functions	Must be on `window.*` — scripts.js is an ES module
Supabase CDN variable	Always use `supabaseClient`, never `supabase` as a local variable name
Table thead z-index	Use `z-[1]`, never `z-10` — navbar owns `z-10`